

# Comprehensive Thesis Design: LLMs × Pepper Robot × Human Experiments

## PART 1: Existing Literature — Social Robots Working with LLMs

Before proposing topics, here is the landscape you should be aware of. This is what exists, and your thesis must go beyond it.

### Key Scientific Papers

| Paper                                                                                              | Venue                        | What They Did                                                             | Gap Remaining                                                                       |
|----------------------------------------------------------------------------------------------------|------------------------------|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Irfan et al. (2024) — "Between Reality and Delusion: Challenges of Applying LLMs to Social Robots" | HRI 2024 Workshop            | Identified challenges of integrating ChatGPT with Pepper; single-LLM only | No multi-LLM comparison, no smart routing, no RAG                                   |
| Nichols et al. (2024) — "Can ChatGPT Control a Pepper Robot Adequately?"                           | arXiv                        | Connected GPT-4 to Pepper for dialogue; evaluated single model            | Only one LLM tested, no systematic comparison, no user study with multiple backends |
| Grassi et al. (2024) — "ChatGPT-based Pepper Robot for Restaurant Service"                         | HAI 2024                     | Pepper as a restaurant assistant using ChatGPT                            | Single LLM, no routing, no domain-specific RAG                                      |
| Spitale et al. (2024) — "Vita: An LLM-Powered Social Robot for Wellbeing"                          | arXiv                        | Pepper-like robot for mental wellbeing conversations using LLM            | Single LLM, no model comparison, no evaluation framework                            |
| Billing et al. (2024) — "Language Models for Human-Robot Interaction"                              | Frontiers in Robotics and AI | Survey of LLM use in HRI, proposes design patterns                        | Survey paper, no experiments comparing multiple LLMs in embodied setting            |
| Catania et al. (2023) — "LLM-Enhanced Social Robots for Elderly Care"                              | IEEE                         | Pepper with GPT for elderly companionship                                 | Single LLM, no RAG, no comparison                                                   |
| Wake et al. (2023) — "ChatGPT Empowered Long-Step Robot Control"                                   | arXiv:2304.03893             | GPT generates multi-step robot action plans                               | Not social interaction focused, single LLM                                          |
| Vemprala et al. (2023) — "ChatGPT for Robotics: Design Principles"                                 | Microsoft Research           | Design principles for connecting ChatGPT to robots                        | Framework paper, single model, no HRI user study                                    |
| Kawaharazuka et al. (2024) — "Real-World Robot Applications of Foundation Models: A Review"        | arXiv                        | Comprehensive review of foundation models in robotics                     | Review, identifies need for multi-model comparison in embodied settings             |

## Key GitHub Repositories and Resources

| Repository / Resource                             | What It Contains                                        |
|---------------------------------------------------|---------------------------------------------------------|
| <a href="#"><u>Awesome-LLM-Robotics</u></a>       | Curated collection of 200+ papers on LLMs + robotics    |
| <a href="#"><u>PromptCraft-Robotics</u></a>       | Microsoft's framework for ChatGPT + robot control       |
| <a href="#"><u>LiteLLM</u></a>                    | Unified API gateway for 100+ LLM providers              |
| <a href="#"><u>LangChain</u></a>                  | RAG pipelines, document loaders, chains                 |
| <a href="#"><u>LangGraph</u></a>                  | Stateful agent graphs with conditional routing          |
| <a href="#"><u>SoftBank Robotics Labs</u></a>     | Official Pepper/NAO community examples                  |
| <a href="#"><u>pepper-chatgpt-integration</u></a> | Various community projects connecting Pepper to ChatGPT |
| <a href="#"><u>Awesome-LLM-Ensemble</u></a>       | Multi-model consensus and ensemble methods              |
| <a href="#"><u>ChromaDB</u></a>                   | Vector store for RAG                                    |
| <a href="#"><u>LMSYS Chatbot Arena</u></a>        | Human preference ELO leaderboard for LLMs               |

## Blog Posts and Practical Resources

- HuggingFace Blog: "DeepSeek R1 Controls Robotic Arm" — open-source LLM controlling a physical robot
- Latitude AI Blog: "LLM Routing Cost Reduction in RAG" — 75% cost savings with routing
- SoftBank Developer Portal: NAOqi SDK documentation for Pepper
- Ollama Blog: Running local models for robotics applications

## The Critical Gap You Will Fill

### What NO ONE has done:

1. No study has systematically compared multiple LLMs as interchangeable backends for a social robot through human experiments
2. No work has applied smart routing between LLMs in an embodied social robot context
3. No research has used RAG-augmented social robots to compare LLM performance on domain-specific knowledge tasks with real users
4. No study has compared LLM rankings derived from embodied HRI with standard text-only benchmark rankings
5. No work has combined LangGraph-based agent orchestration with social robot interaction management

---

## PART 2: Thesis Topic Proposals

### TOPIC 1 (★ PRIMARY RECOMMENDATION)

**Title:** *"Embodied LLM Arena: Multi-Model Smart Routing and RAG-Augmented Knowledge Grounding for Task-Based Human-Robot Interaction with Pepper"*

**Research Gap:**

Current LLM evaluations (MMLU, Chatbot Arena, LiveBench) are entirely text-based. Social robot studies that use LLMs connect only a single model (typically ChatGPT). No study has: (a) compared multiple LLMs as interchangeable backends for a social robot, (b) applied dynamic smart routing between models during live HRI, or (c) evaluated whether LLM rankings change when evaluated through embodied interaction rather than text-only benchmarks. Furthermore, no RAG-augmented social robot has been tested with multiple LLM backends to compare domain-specific knowledge delivery performance.

### **Hypothesis:**

- H1: LLM quality rankings obtained through embodied human-robot interaction will differ significantly from text-only benchmark rankings, because physical presence, latency sensitivity, and multimodal feedback change how humans perceive response quality.
- H2: Dynamic smart routing between LLMs based on task type and language will produce higher user satisfaction than any single fixed-model configuration.
- H3: RAG-augmented responses will be rated significantly more accurate and trustworthy than non-RAG responses across all LLM backends.

### **Experimental Design:**

*Participants:* 15–25 people (lab visitors, students, staff — the people whose information is in the knowledge base)

*Independent Variables:*

- LLM Backend: (a) GPT-4o-mini, (b) Claude Haiku, (c) Gemini Flash, (d) DeepSeek-V3, (e) Local Ollama model (Llama3 or Qwen2.5), (f) Smart-Routed (OmniLLM selects dynamically)
- RAG Condition: RAG-on vs. RAG-off
- Language: The participant's native language (system auto-detects and adapts)

*Task Types (each participant does all 4):*

- T1 — Information Retrieval: "What are the lab opening hours?" / "Tell me about Professor X's research" (tests RAG + factual accuracy)
- T2 — Guidance/Navigation: "Where is Room 305?" / "Guide me to the cafeteria" (tests robot gestures + pointing + LLM response)
- T3 — Social Conversation: Open-ended chat about the visitor's interests (tests fluency, empathy, naturalness)
- T4 — Multilingual Task: Repeat one task in a different language (tests multilingual capability)

*Dependent Variables:*

- Task completion success (binary)
- Response latency (measured automatically)
- Factual accuracy (RAG faithfulness, scored by LLM-as-Judge via OmniLLM)
- User satisfaction (post-interaction Likert-scale questionnaire)
- Perceived intelligence, naturalness, trust (Godspeed questionnaire or custom HRI scales)

- Gesture appropriateness (observer-rated)

*Design:* Within-subjects, counterbalanced (Latin square). Each participant interacts with Pepper powered by 3 different LLM conditions (randomized assignment). After each interaction block, they fill out a short questionnaire. At the end, they do a pairwise preference comparison ("Which Pepper did you prefer?").

#### **Tools / Models Needed:**

- Pepper robot (NAOqi, Python 2.7 client)
- OmniLLM platform (your existing project)
- LiteLLM for unified LLM gateway
- LangChain + ChromaDB for RAG pipeline
- LangGraph for conversation state management and routing logic
- Ollama for local models (Llama3:8b, Qwen2.5:7b, Mistral:7b)
- Cloud APIs: OpenAI (GPT-4o-mini), Anthropic (Claude Haiku), Google (Gemini Flash), DeepSeek
- Flask/FastAPI for Python 2.7 ↔ 3.x HTTP bridge
- Whisper (local or API) for speech-to-text
- Standard questionnaire tools (Google Forms or paper)

#### **Why the Robot Is Essential (Cannot Be Done Without It):**

- The core research question is about *embodied* evaluation — does physical presence change LLM quality perception?
- Gesture-speech coordination timing is a robot-specific variable
- Navigation/pointing tasks require a physical body
- The tablet displays visual information (maps, schedules) synchronized with speech
- Physical presence effects (documented extensively in HRI literature) are the confound that makes this study novel compared to any chatbot study

#### **Risks and Challenges:**

- Pepper's speech recognition may be unreliable → Mitigation: use external microphone + Whisper API
- Python 2.7 ↔ 3.x bridge latency → Mitigation: optimize HTTP calls, use local Flask server
- 1-month timeline is tight → Mitigation: use minimal viable task set, recruit from lab visitors
- Cloud API costs → Mitigation: use mini/haiku/flash tier models + Ollama local for most testing
- Small sample size → Mitigation: use within-subjects design to increase statistical power

#### **Expected Contribution:**

1. First systematic multi-LLM comparison in embodied social robot HRI

2. First evidence on whether embodied LLM rankings differ from text-only rankings
3. First application of smart routing in social robotics
4. First RAG-augmented social robot with multi-model backend comparison
5. Open-source architecture (OmniLLM + Pepper bridge) for reproducibility
6. Novel "Embodied LLM Arena" evaluation methodology

## PART 3: Rankings and Recommendations

### Ranked by Novelty and Impact

| Rank | Topic                             | Novelty (1-10) | Impact (1-10) | Feasibility in 1 Month (1-10) | Overall |
|------|-----------------------------------|----------------|---------------|-------------------------------|---------|
| 1    | Topic 1:<br>Embodied LLM<br>Arena | 10             | 10            | 7                             | ★★★★★   |
|      |                                   |                |               |                               |         |
|      |                                   |                |               |                               |         |
|      |                                   |                |               |                               |         |

### Highest Chance of Publication: Topic 1

It has the broadest contribution (multi-LLM comparison + smart routing + RAG + embodied evaluation methodology), addresses a clear gap, and the results are inherently interesting regardless of outcome. Suitable venues: HRI Conference, ICRA, IEEE RO-MAN, Frontiers in Robotics and AI, MDPI Robotics.

### Which Would Impress OpenAI / DeepMind / Robotics Startups: Topic 1

- **OpenAI/Anthropic:** They care about evaluation methodology. "Embodied LLM Arena" is a novel evaluation paradigm that extends Chatbot Arena to the physical world.
- **DeepMind:** They care about embodied AI. This bridges LLMs and physical robots with rigorous experiments.
- **Robotics startups:** They care about practical multi-LLM architectures for robots. Your OmniLLM + Pepper bridge is directly deployable.

### Which Could Realistically Become a Research Paper: Topic 1

Topic 1 is the full paper.

### Which Builds the Most Valuable Skills: Topic 1

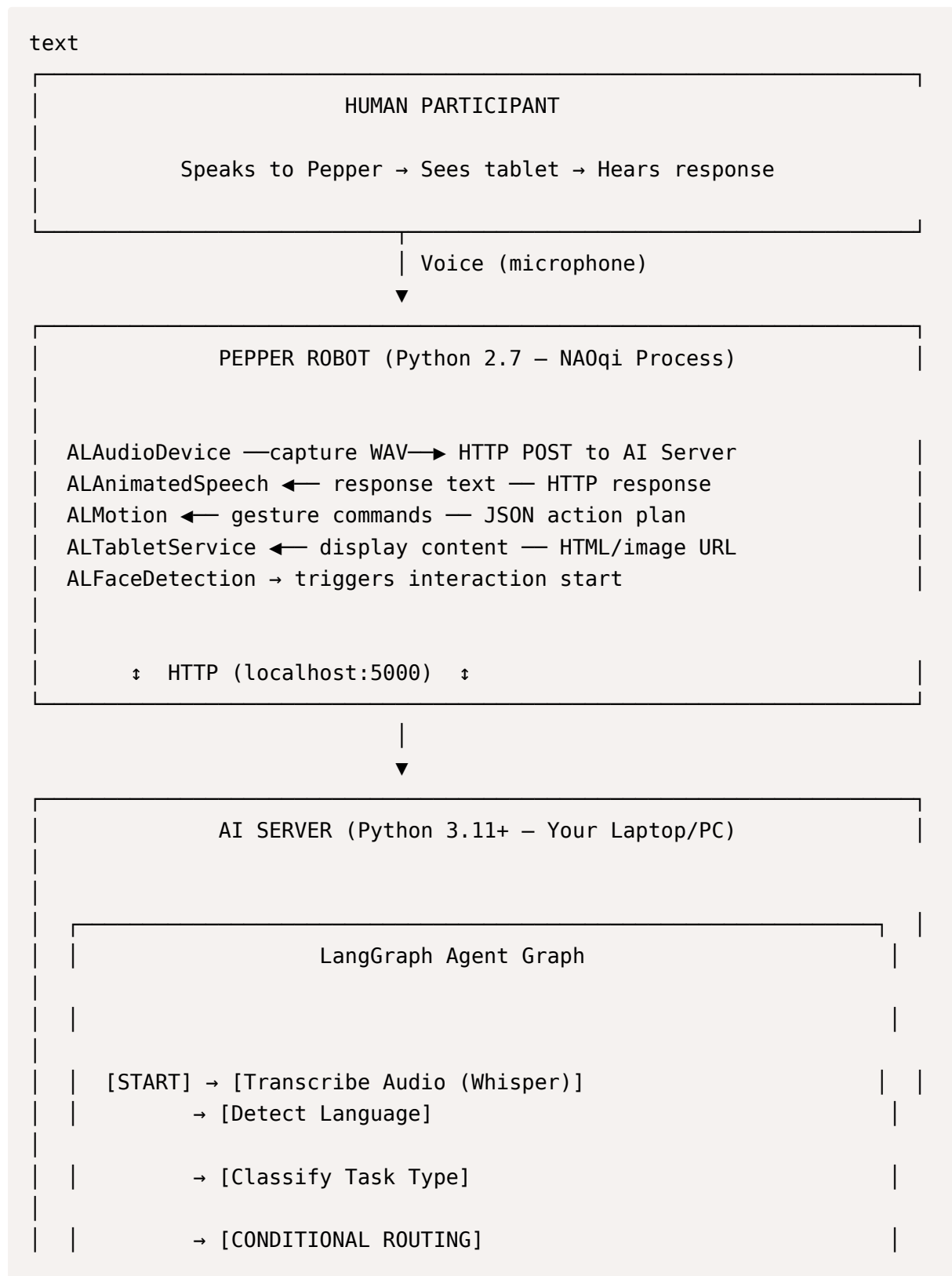
It forces you to master: LLM APIs, RAG pipelines, LangGraph agent design, robot integration, HTTP bridge architecture, experimental design, user studies, statistical analysis — the complete stack an AI/Robotics engineer needs.

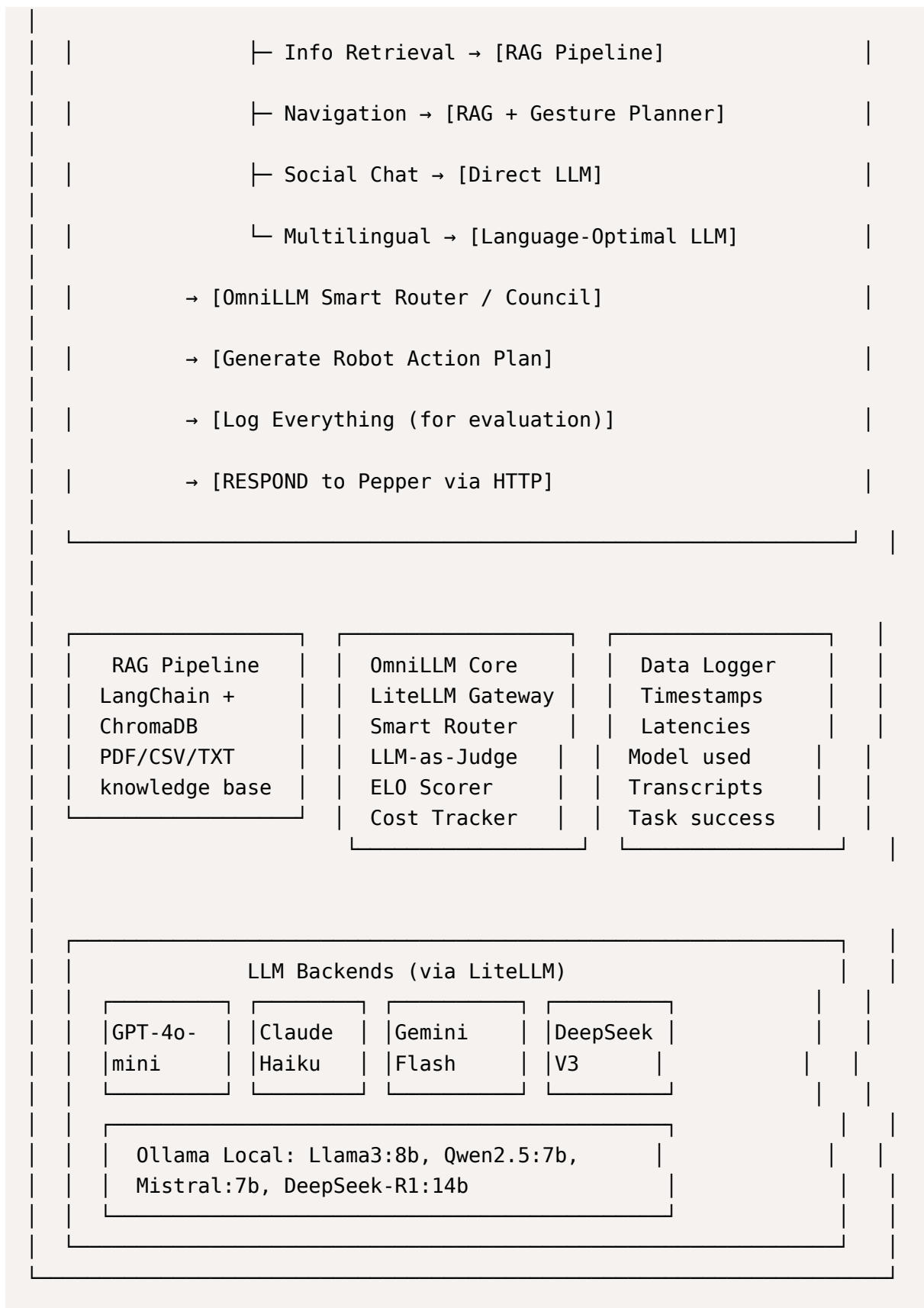
## PART 4: The Recommended Thesis — Full Design

## Title

"Embodied LLM Arena: Multi-Model Smart Routing and RAG-Augmented Knowledge Grounding for Task-Based Human-Robot Interaction with Pepper"

## System Architecture





**How Each Tool Fits In  
OmniLLM (your project):**

- Central orchestration platform
- Smart Router selects which LLM to use based on task type, language, cost, quality history
- LLM-as-Judge evaluates every response automatically (referenceless + reference-based using RAG ground truth)
- ELO Scorer builds a leaderboard from pairwise human preferences
- Cost Tracker monitors spend per model
- Consensus Engine (LLM Council) can be used as one experimental condition

#### LiteLLM:

- Unified API gateway — all LLMs called through identical interface
- Adding/switching models requires zero code changes (just YAML in OmniLLM)
- Handles retries, fallbacks, rate limiting

#### LangChain:

- **Document Loaders:** PyPDFLoader, CSVLoader, TextLoader, Docx2txtLoader — load your knowledge base
- **Text Splitter:** RecursiveCharacterTextSplitter(chunk\_size=500, chunk\_overlap=50) — chunk documents
- **Embeddings:** HuggingFaceEmbeddings (free, local) or OpenAI embeddings
- **Vector Store:** ChromaDB (persistent, local, free) — store and retrieve document chunks
- **RetrievalQA Chain:** Combines retriever + LLM to answer questions from knowledge base
- **ConversationBufferMemory:** Maintains multi-turn conversation history per session

#### LangGraph:

- **Conversation State Machine:** Manages the entire interaction flow as a directed graph
- **Nodes:** transcribe → detect\_language → classify\_task → route\_to\_pipeline → generate\_response → create\_robot\_action → log\_data
- **Conditional Edges:** Based on task classification, route to RAG pipeline or direct LLM or consensus engine
- **State:** Carries conversation history, detected language, current task, selected model, all intermediate results
- **Human-in-the-Loop:** Can pause for human confirmation at any node (useful for navigation commands)

#### OpenClaw / Agent Frameworks:

- If OpenClaw is an agent orchestration tool you have access to, it can serve as an alternative to LangGraph for managing the multi-step interaction pipeline
- It could orchestrate the multi-agent workflow: one agent for speech processing, one for task classification, one for RAG retrieval, one for response generation, one for robot action planning

- If OpenClaw supports tool-calling, it can call OmniLLM's smart router and RAG pipeline as tools

#### Ollama:

- Free, local LLM inference — no API costs
- Models: Llama3:8b (general English), Qwen2.5:7b (multilingual, strong Chinese), Mistral:7b (strong European languages), DeepSeek-R1:14b (reasoning)
- Critical for the "local vs. cloud" comparison dimension
- Also used as a free LLM-as-Judge when you don't want to pay for GPT-4o evaluation

#### Whisper:

- Speech-to-text transcription
- Provides language detection for free (returns detected language with transcription)
- Can run locally (whisper.cpp or faster-whisper) for zero cost
- Or use OpenAI Whisper API for higher accuracy

### Knowledge Base Design (What to Feed the RAG)

| Document Type                           | Content                                                           | Purpose                                                                  |
|-----------------------------------------|-------------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>Visitor profiles</b> (CSV/JSON)      | Name, role, department, research interests, scheduled visit time  | Pepper can greet visitors by name, answer questions about their schedule |
| <b>Lab information</b> (PDF/TXT)        | Room locations, equipment, opening hours, safety rules, wifi info | Information retrieval tasks                                              |
| <b>Research papers</b> (PDF)            | Lab publications, project descriptions                            | Domain expertise questions                                               |
| <b>Event schedules</b> (CSV)            | Seminars, meetings, deadlines                                     | Calendar/scheduling tasks                                                |
| <b>University map</b> (TXT description) | Building layout, directions to rooms                              | Navigation/guidance tasks                                                |
| <b>FAQ document</b> (TXT)               | Common questions and answers                                      | Baseline for evaluation                                                  |

### The 4 Task Types in Detail

#### Task 1 — Information Retrieval (RAG-dependent):

- "What time does Professor X arrive today?"
- "Tell me about the ongoing research projects in this lab."
- "What is the wifi password?"
- *Why robot is essential:* Pepper greets the person, uses tablet to show additional info, uses gestures while explaining.

#### Task 2 — Navigation/Guidance (Robot body essential):

- "Where is Room 305?"

- "Can you point me to the cafeteria?"
- "How do I get to the seminar room?"
- *Why robot is essential:* Pepper physically points in directions, shows a map on its tablet, can walk partway to guide the person. A chatbot cannot point or show physical direction.

### Task 3 — Social Conversation (Tests natural dialogue):

- Open-ended: "How are you?" / "Tell me something interesting" / "What do you think about AI?"
- *Why robot is essential:* Physical presence creates social pressure and engagement that text chat doesn't. Eye contact, head tracking, animated gestures change perceived quality.

### Task 4 — Multilingual Interaction (Tests language routing):

- Repeat one of the above tasks in a different language
- System auto-detects language via Whisper and routes accordingly
- *Why robot is essential:* The robot's physical embodiment makes the multilingual experience more immersive and testable than a screen chatbot.

## Experimental Conditions

| Condition                     | Description                         | What It Tests                   |
|-------------------------------|-------------------------------------|---------------------------------|
| <b>A: Fixed Cloud LLM</b>     | GPT-4o-mini for all tasks           | Baseline cloud performance      |
| <b>B: Fixed Local LLM</b>     | Llama3:8b (Ollama) for all tasks    | Baseline local/free performance |
| <b>C: Smart-Routed</b>        | OmniLLM selects best model per task | Smart routing effectiveness     |
| <b>D: Consensus (Council)</b> | 3 models answer, best synthesized   | Consensus vs. single model      |
| <b>E: RAG-Off Control</b>     | Same as A but without RAG           | Isolates RAG contribution       |

*Each participant experiences 3 conditions (counterbalanced). Total: ~20 participants × 3 conditions × 4 tasks = 240 interaction data points.*

## Metrics and Data Collection

### Automatic (logged by the system):

- Which LLM was used for each response
- Response latency (time from end of speech to start of robot response)
- Token count (input and output)
- Cost per interaction
- RAG retrieval scores (relevance of retrieved chunks)
- LLM-as-Judge scores (OmniLLM evaluates each response on: accuracy, relevance, fluency, safety)
- Language detected
- Task classification

### Human-Rated (questionnaire after each condition):

- "The robot's answers were accurate" (1–7 Likert)
- "The robot was natural to talk to" (1–7 Likert)
- "I trust the information the robot gave me" (1–7 Likert)
- "The robot's gestures were appropriate" (1–7 Likert)
- "The robot responded quickly enough" (1–7 Likert)
- Pairwise preference: "Which version of Pepper did you prefer?" (for ELO)

### Observer-Rated (you or a colleague watching):

- Gesture-speech synchronization quality
- Task completion (did the person get the correct information?)
- Interaction breakdown moments

---

## How to Automatically Rate/Grade the LLMs (Your "Last Part" Idea)

This is where OmniLLM's evaluation framework shines. Here is what the system does automatically after every interaction:

1. **Referenceless Evaluation:** OmniLLM sends the prompt + response to a judge LLM (GPT-4o-mini) and asks it to rate on coherence, helpfulness, fluency, safety (1–10 each).
2. **Reference-Based Evaluation:** For RAG tasks, the retrieved document chunks serve as the reference. The judge evaluates whether the response faithfully uses the retrieved information (RAG faithfulness score).
3. **Pairwise Evaluation (ELO):** When participants state their preference between conditions, OmniLLM's ELO scorer updates model ratings. After all experiments, you get an "Embodied LLM Leaderboard."
4. **Automatic Hallucination Detection:** Compare the response against the RAG knowledge base. If the response contains claims not supported by retrieved documents, flag as hallucination. Count hallucination rate per model.
5. **Latency-Quality Composite Score:** OmniLLM computes:  $\text{value} = \text{quality} \times 0.5 + (1/\text{latency}) \times 0.3 + (1/\text{cost}) \times 0.2$  — which model gives the best quality-per-second-per-dollar in the embodied setting?
6. **Cross-Reference with Standard Benchmarks:** Take each model's MMLU/Chatbot Arena score and correlate with your embodied scores. The discrepancy IS your key finding.

---

## Where OpenClaw Fits

If OpenClaw is an agent orchestration framework you have access to:

- Use it to define the multi-step agent pipeline: perceive → classify → retrieve → generate → act
- It can manage tool-calling: the agent calls RAG retrieval as a tool, calls OmniLLM router as a tool, calls robot action API as a tool

- If OpenClaw supports multi-agent patterns, you can have: a "Retrieval Agent" (handles RAG), a "Router Agent" (selects LLM), a "Robot Agent" (generates action plans), and a "Judge Agent" (evaluates quality)
  - This is an alternative to LangGraph — use whichever you're more comfortable with
  - If OpenClaw has its own LLM routing capabilities, you can compare OpenClaw routing vs. OmniLLM routing as an experimental condition
- 

## PART 5: One-Month Execution Plan

### Week 1: Infrastructure Setup (Days 1–7)

#### Day 1–2: Pepper ↔ AI Server Bridge

- Set up Flask server on your laptop (Python 3.11)
- Create HTTP endpoints: /transcribe (receives WAV, returns text), /respond (receives text, returns response + action JSON)
- On Pepper (Python 2.7 NAOqi): write a simple client that captures audio, sends to Flask, receives response, speaks it, executes gestures
- Test with a hardcoded response first, then with a single LLM

#### Day 3–4: OmniLLM + LiteLLM Integration

- Configure your models.yaml with all target models (GPT-4o-mini, Claude Haiku, Gemini Flash, DeepSeek, + 3 Ollama models)
- Test that OmniLLM can call all models and get responses
- Set up smart router with routing rules
- Test the LLM-as-Judge pipeline

#### Day 5–6: RAG Pipeline

- Create your knowledge base documents (visitor profiles, lab info, room locations, schedules)
- Set up LangChain: document loaders → text splitter → HuggingFace embeddings → ChromaDB
- Build RetrievalQA chain
- Test: ask questions about the knowledge base, verify correct retrieval

#### Day 7: LangGraph Agent + End-to-End Integration

- Build the LangGraph state machine (transcribe → classify → route → RAG/LLM → action plan → respond)
- Connect everything: Pepper → Flask → LangGraph → OmniLLM → LLM → response → Pepper
- End-to-end smoke test: speak to Pepper, get an LLM-powered RAG-augmented response with gestures

## **Week 2: Refinement and Experiment Preparation (Days 8–14)**

### **Day 8–9: Refine Robot Behaviors**

- Map response types to Pepper gestures (information → open palms, pointing → arm raise toward direction, greeting → wave)
- Set up tablet displays (show retrieved info, maps, schedules)
- Tune Whisper for your acoustic environment
- Test multilingual detection and routing

### **Day 10–11: Experiment Protocol Design**

- Write the experiment script (what the experimenter says/does)
- Create the task prompts for each task type
- Design the questionnaire (Google Forms or paper)
- Create consent forms
- Set up automatic logging (every interaction logged with timestamp, model used, latency, transcripts, scores)

### **Day 12–13: Pilot Testing**

- Run 3–4 pilot sessions with lab colleagues
- Identify and fix issues (latency, speech recognition errors, gesture timing)
- Refine task prompts based on pilot feedback
- Verify all conditions switch correctly

### **Day 14: Final Preparations**

- Schedule participants for Week 3
- Prepare the physical lab space
- Create a backup plan (if cloud API fails, fall back to Ollama)
- Finalize all logging and data collection scripts

## **Week 3: Run Experiments (Days 15–21)**

### **Day 15–21: Participant Sessions**

- Run 3–4 participants per day
- Each session: ~20–30 minutes (brief intro, 3 conditions × 4 tasks, questionnaire, debrief)
- Target: 15–20 participants minimum
- Experimenter notes observations during each session
- Back up data daily

## **Week 4: Analysis and Writing (Days 22–30)**

### **Day 22–24: Data Analysis**

- Aggregate automatic metrics (latency, LLM-as-Judge scores, cost, hallucination rate)
- Aggregate human ratings (questionnaire responses)
- Compute ELO rankings from pairwise preferences
- Statistical tests: repeated-measures ANOVA or Friedman test for condition comparisons
- Generate the "Embodied LLM Leaderboard" and compare with text-only benchmarks

### **Day 25–28: Write Thesis/Paper**

- Introduction + Related Work (leverage OmniLLM's existing literature review)
- System Architecture (the diagrams above)
- Experimental Design
- Results + Discussion
- Conclusions + Future Work

### **Day 29–30: Polish and Submit**

- Create figures and tables
- Proofread
- Prepare code repository for open-source release
- Record a demo video

## **PART 7: Summary of Direct Answers to Your Questions**

### **Q: Can I use OpenClaw somewhere in this process?**

Yes — as an agent orchestration layer, either alongside or instead of LangGraph. Use it to define the multi-step pipeline (perceive → classify → retrieve → generate → act) and to manage tool-calling (RAG retrieval, LLM routing, robot actions as tools).

### **Q: How can I use different LLMs (GPT, Claude, Gemini, DeepSeek, Ollama) with old Pepper?**

Through the Python 2.7 ↔ 3.x HTTP bridge. Pepper (Python 2.7) sends audio/text to a Flask server (Python 3.x). The Flask server calls OmniLLM/LiteLLM, which provides a unified interface to all LLMs. Switching models requires changing one parameter — no Pepper code changes.

### **Q: How can I use LangChain/LangGraph/RAG with Pepper and people?**

LangChain builds the RAG pipeline (load docs → embed → store → retrieve). LangGraph manages the conversation flow as a state machine with conditional routing. The RAG-augmented response is sent back to Pepper via HTTP. People interact naturally with Pepper; they don't know or care about the backend architecture.

### **Q: What about feeding PDFs/papers/READMEs to make the robot domain-specific?**

This is exactly what the RAG pipeline does. Use LangChain's PyPDFLoader, CSVLoader, TextLoader to ingest documents. ChromaDB stores the vectors. When a person asks a question, relevant chunks are retrieved and passed to the LLM as context. The robot answers based only on the provided knowledge. You can test accuracy and hallucination rate per LLM.

---

## Final Recommendation

**Go with Topic 1: "Embodied LLM Arena."** It is the most novel, the most impactful, the most publishable, and it builds the most valuable skills. It directly extends your OmniLLM project into the physical world, fills a clear gap in the literature, and produces results that are interesting regardless of outcome (whether embodied rankings match text-only rankings or not, both findings are publishable).

The minimal first-month prototype is: Pepper + 3 LLM backends (one cloud, one local, one smart-routed) + RAG knowledge base + 4 task types + 15 participants + automatic and human evaluation. This is tight but absolutely doable given your existing OmniLLM infrastructure.